상태 관리와 조건부 분기 (실습)

3-1. 상태 업데이트 규칙 (실습)
3-2. 조건부 엣지 (실습)
3-3. 반복 루프 만들기 (실습)

3-1. 상태 업데이트 규칙 (실습)

> 노드가 반환한 값이 State에 합쳐지는 규칙을 reducer로 제어해봅니다.

2번째 세션에서 본 병합의 기본 동작은 같은 키면 덮어쓰기였습니다.

그런데 대화 기록(messages) 같은 리스트는 노드가 실행될 때마다 누적되어야 합니다. 덮어써지면 앞의 대화가 사라집니다. 그래서 reducer라는 개념이 있습니다.

Reducer는 새 값이 왔을 때 기존 State의 값과 어떻게 합칠지 정하는 규칙입니다. 필드에 추가 정보를 달아 주는 `Annotated` 타입 힌트로 지정합니다. Java로 치면 필드에 붙이는 어노테이션으로 병합 전략을 지정하는 셈입니다.

```
from typing import TypedDict, Annotated
from operator import add

class State(TypedDict):
    answer: str                          # reducer 없음: 덮어쓰기 (기본)
    history: Annotated[list, add]        # add reducer: 이전 리스트 + 새 리스트로 누적
```

가장 자주 쓰는 reducer는 대화 기록 전용인 `add_messages` 입니다. 단순히 이어 붙이기만 하지는 않습니다. 새 ID의 메시지면 뒤에 추가하고 같은 ID면 기존 것을 바꿉니다. `RemoveMessage` 가 들어오면 그 메시지를 지웁니다.

Reducer	동작	주 용도
(없음, 기본)	덮어쓰기	단일 값 (answer, query 등)
`operator.add`	리스트 이어붙이기	로그, 이력 누적
`add_messages`	메시지 추가 + ID 기반 갱신, 삭제	대화 기록 (가장 자주 씀)
직접 만든 함수	자유	dict 깊은 병합 등 특수한 경우

따라하기

1. `add_messages` 의 누적 동작을 확인합니다. 아래 코드를 `day3_03_reducer.py` 로 저장해 실행합니다.

```
# day3_03_reducer.py - add_messages reducer 동작 확인
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_core.messages import HumanMessage, AIMessage


class ChatState(TypedDict):
    messages: Annotated[list, add_messages]


def chat(state: ChatState) -> dict:
    # 새 메시지를 리스트로 반환만 하면 add_messages가 알아서 누적합니다
    return {"messages": [AIMessage(content="안녕하세요, 반갑습니다.")]}


builder = StateGraph(ChatState)
builder.add_node("chat", chat)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile()

# 1차 실행
result = graph.invoke({"messages": [HumanMessage(content="안녕하세요?")]})
print("1차 실행 후 메시지 개수:", len(result["messages"]))

# 2차 실행: 이전 messages를 그대로 이어서 넘기면 계속 누적됩니다
result2 = graph.invoke(
    {"messages": result["messages"] + [HumanMessage(content="잘 지내시나요?")]}
)
print("2차 실행 후 메시지 개수:", len(result2["messages"]))
for m in result2["messages"]:
    print(f"  [{type(m).__name__}] {m.content}")
```

2. 출력을 관찰합니다. 노드는 새 메시지 1개만 반환했는데 State에는 기존 기록 위에 쌓였습니다. 노드는 추가분만 신경 쓰고 합치는 방법은 State 스키마가 책임집니다.

막힐 때

- `add_messages` 를 붙였는데 메시지가 1개만 남습니다.

  `messages: list` 라고만 쓰면 기본 동작(덮어쓰기)이 적용됩니다. `Annotated[list, add_messages]` 문법과 import 경로를 확인합니다.

- import 경로가 헷갈립니다.

  `from langgraph.graph.message import add_messages`가 표준입니다.

- reducer가 붙은 필드에 전체 기록을 다시 반환했더니 기록이 두 배로 불어납니다.

  reducer가 붙은 필드에는 추가분만 반환해야 합니다.

확인: 1차 실행 후 2개, 2차 실행 후 4개로 메시지가 누적되면 성공입니다.

> 합치는 방법은 노드가 아니라 State 스키마의 reducer가 정한다는 규칙을 익혔습니다.

3-2. 조건부 엣지 (실습)

> 상태 값에 따라 다른 노드로 분기하는 그래프를 만들어봅니다.

조건부 엣지는 라우팅 함수로 만듭니다. 라우팅 함수는 State를 받아 다음 노드의 이름(문자열)을 반환하는 함수입니다. 이것이 조건부 엣지의 전부입니다.

시나리오는 사용자 입력이 질문인지 감탄인지 분류해서 다른 노드로 보내는 그래프입니다. 질문이면 LLM이 답하고 감탄이면 LLM 없이 정해진 반응을 합니다.

따라하기

1. 분기 그래프를 작성해 `day3_04_conditional.py` 로 저장하고 실행합니다. Bedrock은 응답의 content를 문자열이 아니라 블록 리스트로 주기도 합니다. 그래서 텍스트만 뽑아 주는 `get_text` 함수를 파일 앞부분에 하나 두고 씁니다.

```
# day3_04_conditional.py - 조건부 엣지로 분기하기
from typing import TypedDict, Literal
from dotenv import load_dotenv
from langgraph.graph import StateGraph, START, END
from langchain_aws import ChatBedrockConverse

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


class ClassifyState(TypedDict):
    user_input: str
    intent: str
    response: str


def classify(state: ClassifyState) -> dict:
    """입력에 '?'가 있으면 question, 없으면 exclaim으로 분류합니다."""
    intent = "question" if "?" in state["user_input"] else "exclaim"
    return {"intent": intent}


def answer_question(state: ClassifyState) -> dict:
    """질문에 LLM으로 답합니다."""
    response = llm.invoke(f"다음 질문에 한 줄로 답하세요: {state['user_input']}")
    return {"response": get_text(response)}


def react_exclaim(state: ClassifyState) -> dict:
    """감탄에는 LLM 없이 정해진 반응을 합니다."""
    return {"response": f"'{state['user_input']}' 라니 멋진 일이네요."}


def route_by_intent(state: ClassifyState) -> Literal["answer_question", "rea
```

```
ct_exclaim"]:
    """라우팅 함수: State를 보고 다음 노드의 이름을 반환합니다."""
    if state["intent"] == "question":
        return "answer_question"
    return "react_exclaim"


builder = StateGraph(ClassifyState)
builder.add_node("classify", classify)
builder.add_node("answer_question", answer_question)
builder.add_node("react_exclaim", react_exclaim)

builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route_by_intent)   # 분기 등록
builder.add_edge("answer_question", END)
builder.add_edge("react_exclaim", END)
graph = builder.compile()

for text in ["오늘 사내 식당 메뉴 뭐야?", "와 배포가 한 번에 성공했다", "LangGraph가 뭐
야?"]:
    result = graph.invoke({"user_input": text, "intent": "", "response":
""})
    print(f"입력: {text}")
    print(f"분류: {result['intent']} / 응답: {result['response']}\n")
```

2. 코드에서 두 가지를 관찰합니다.
   - 라우팅 함수의 반환 타입에 `Literal["answer_question", "react_exclaim"]` 을 썼습니다. 이 함수가 두 노드로만 보낸다는 선언이라 IDE가 오타에 바로 빨간 줄을 그어 줍니다. 함수 선언부만 봐도 분기 목적지를 알 수 있습니다. Java의 enum 반환과 비슷한 효과입니다.
   - 판단과 목적지를 분리하는 매핑 방식도 있습니다. 라우팅 함수는 True/False만 반환하고 노드 이름 매핑은 바깥에서 dict로 넘깁니다.

```
def is_question(state: ClassifyState) -> bool:
    return state["intent"] == "question"

builder.add_conditional_edges(
    "classify",
    is_question,
    {True: "answer_question", False: "react_exclaim"},  # 길 안내표
)
```

막힐 때

- 라우팅 함수를 `add_node` 로 등록했더니 이상하게 동작합니다.

  라우팅 함수는 노드가 아닙니다. `add_conditional_edges` 의 두 번째 인자로만 넘깁니다. 노드는 일을 하고 라우팅 함수는 길만 안내한다고 구분하면 됩니다.

- 라우팅 함수가 존재하지 않는 노드 이름을 반환해 실행 중 에러가 납니다.

  Literal 타입 힌트를 쓰면 이런 실수를 코드 작성 시점에 잡을 수 있습니다.

- 분기가 두 경로 모두 실행됩니다.

  `add_conditional_edges` 를 등록한 노드에 `add_edge` 로 일반 엣지를 또 건 경우입니다. 한 노드의 출구는 한 가지 방식으로만 정의합니다.

확인: 물음표가 있는 입력은 question으로 분류되어 LLM 답변이 나오고 없는 입력은 exclaim으로 분류되어 정해진 반응이 나오면 성공입니다.

> 라우팅 함수의 반환값이 곧 다음 노드의 이름이라는 조건부 분기 원리를 익혔습니다.

3-3. 반복 루프 만들기 (실습)

> 조건을 만족할 때까지 이전 노드로 되돌아가는 반복 그래프를 만들어봅니다.

조건부 엣지의 목적지로 이전 노드를 지정하면 그것이 곧 반복(Cycle, 사이클)입니다. 별도 문법이 없습니다.

시나리오는 답변을 생성하고 품질이 기준에 못 미치면 다시 생성하는 그래프입니다.

따라하기

1. 반복 그래프를 작성해 `day3_05_cycle.py` 로 저장하고 실행합니다.

```python
# day3_05_cycle.py - 조건을 만족할 때까지 반복하는 그래프
from typing import TypedDict
from dotenv import load_dotenv
from langgraph.graph import StateGraph, START, END
from langchain_aws import ChatBedrockConverse

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinstance(block, dict))
    return content


class QualityState(TypedDict):
    question: str
    answer: str
    attempts: int
    quality_score: float


def generate(state: QualityState) -> dict:
    """답변을 생성하고 시도 횟수를 1 올립니다."""
    response = llm.invoke(
        f"다음 질문에 200자 이상으로 자세하게 답하세요: {state['question']}"
    )
    return {
        "answer": get_text(response),
        "attempts": state.get("attempts", 0) + 1,
    }


def check_quality(state: QualityState) -> dict:
    """품질 평가. 실습용으로 단순하게 길이 기반 점수를 씁니다 (200자면 만점)."""
    score = min(len(state["answer"]) / 200, 1.0)
    return {"quality_score": score}
```

```
def route_quality(state: QualityState) -> str:
    """품질이 좋거나 시도 한도에 도달하면 종료하고 아니면 재시도합니다."""
    if state["quality_score"] >= 0.8:
        return END
    if state["attempts"] >= 3:       # 안전장치: 무한 루프 방지
        return END
    return "generate"                # 이전 노드로 돌아감 = 반복


builder = StateGraph(QualityState)
builder.add_node("generate", generate)
builder.add_node("check_quality", check_quality)
builder.add_edge(START, "generate")
builder.add_edge("generate", "check_quality")
builder.add_conditional_edges("check_quality", route_quality)
graph = builder.compile()

result = graph.invoke({
    "question": "LangGraph의 장점을 설명해 주세요",
    "answer": "",
    "attempts": 0,
    "quality_score": 0.0,
})
print(f"시도 횟수: {result['attempts']}")
print(f"품질 점수: {result['quality_score']:.2f}")
print(f"답변 길이: {len(result['answer'])}자")
print(f"답변 미리보기: {result['answer'][:80]}...")
```

2. 모델이 처음부터 긴 답을 주면 1회로 끝나고 짧은 답을 주면 attempts가 2, 3으로 올라갑니다. 재시도를 강제로 관찰하려면 generate의 프롬프트에서 "200자 이상으로 자세하게"를 "한 문장으로 짧게"로 바꿔 다시 실행해 봅니다.

반복 그래프에는 안전장치가 두 개 필요합니다. 하나는 State의 시도 횟수입니다. LLM이 끝까지 조건을 만족하지 못할 수 있으므로 매번 검사해 빠져나오게 합니다.

다른 하나는 LangGraph가 기본으로 걸어 둔 25단계 제한(`recursion_limit`)입니다. 이를 넘으면 `GraphRecursionError` 가 납니다.
정상적인 반복인데 한도가 부족하면 `graph.invoke(..., config={"recursion_limit": 50})` 처럼 늘릴 수 있습니다.
다만 이 에러가 났다면 대부분은 한도를 늘릴 것이 아니라 탈출 조건이 빠졌는지 먼저 의심해야 합니다.

막힐 때

- `attempts`가 계속 1입니다.

  generate 노드에서 `state.get("attempts", 0) + 1` 을 반환하는지 확인합니다. attempts에는 reducer가 없으므로 노드가 명시적으로 더한 값을 반환해야 갱신됩니다.

- 라우팅 함수에서 `"END"` 문자열을 반환했더니 에러가 납니다.

  `END`는 `from langgraph.graph import END`로 가져오는 상수입니다. 따옴표를 붙이면 그 이름의 노드를 찾다가 실패합니다.

확인: 프롬프트를 "한 문장으로 짧게"로 바꿨을 때 시도 횟수가 2 이상으로 올라가는 것을 보면 성공입니다.

> 조건부 엣지의 목적지를 이전 노드로 지정하면 반복이 되고 반복에는 반드시 탈출 조건이 필요하다는 원칙을 익혔습니다.
